

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторне заняття №2
з навчальної дисципліни
"Технології розробки корпоративних Web-додатків"

Виконала:

студент 401-ТН

Стефанович Богдан Ігорович

Перевірив:

Тютюнник Петро Богданович

Полтава – 2026

Тема: Проектування архітектури програмного забезпечення та моделювання даних

Завдання:

1. Обґрунтування вибору технологічного стеку (Technology Stack)

На основі вимог, зібраних у першій лабораторній роботі, оберіть технології для реалізації проєкту.

- **Створіть документ або розділ у звіті, де чітко визначте:**
 - **Мова програмування Back-end:** (наприклад, C#/.NET, Java/Spring, Node.js, Python/Django).
 - **Фреймворк Front-end:** (наприклад, React, Angular, Vue.js або Server-Side Rendering — Razor/Thymeleaf).
 - **Система керування базами даних (СКБД):** (PostgreSQL, MS SQL, MongoDB, Redis тощо).
 - **Інструменти DevOps/Hosting:** (Docker, GitHub Actions, Azure/AWS — опціонально).
- **Надайте обґрунтування:** Чому саме ці технології підходять для вирішення вашої бізнес-задачі? (Наприклад: "Обрано MongoDB, оскільки структура даних каталогу товарів є гнучкою та часто змінюється" або "Обрано .NET через надійну типізацію та швидкість розробки корпоративних API").

2. Архітектурне моделювання за методологією C4 (C4 Model)

Спроектуйте архітектуру вашої системи, створивши три рівні діаграм C4. Це допоможе зрозуміти структуру системи від загального до конкретного.

- **Рівень 1: System Context Diagram (Контекст).**
 - Покажіть вашу систему як одну "чорну скриньку" в центрі.
 - Відобразіть усіх користувачів (Actors) та зовнішні системи (наприклад, платіжні шлюзи, API поштових розсилок), з якими вона взаємодіє.

- **Рівень 2: Container Diagram (Контейнери).**

- "Відкрийте" систему і покажіть її основні технічні блоки (контейнери).
- Це можуть бути: Single Page Application, Mobile App, API Application, Database, File System.
- Вкажіть технології для кожного контейнера та протоколи взаємодії між ними (HTTP/HTTPS, TCP, GRPC).

- **Рівень 3: Component Diagram (Компоненти).**

- Деталізуйте **один** основний контейнер (наприклад, API Application).
- Покажіть внутрішні компоненти (наприклад: AuthController, EmailService, UserRepository) та їхні зв'язки.

3. Проєктування бази даних (ER-Model / Data Schema) Спроєктуйте схему зберігання даних. Вибір типу діаграми залежить від обраної СКБД:

- **Варіант А: Реляційна база даних (SQL)**

- Побудуйте класичну **ER-діаграму (Entity-Relationship Diagram)**.
- Вкажіть сутності, атрибути, первинні (PK) та зовнішні (FK) ключі.
- Визначте типи зв'язків (1:1, 1:N, M:N).

- **Варіант Б: Нереляційна база даних (NoSQL)**

- Створіть діаграму або схему колекцій/документів.
- Відобразіть структуру JSON-документів для основних сутностей.
- Вкажіть стратегію зв'язків: **Embedding** (вкладення даних) чи **Referencing** (посилання на ID).
- Обґрунтуйте вибір стратегії (наприклад, "Використано Embedding для коментарів, щоб отримати пост разом із коментарями за один запит").

Хід роботи

1. Обґрунтування вибору технологічного стеку (Technology Stack).

Back-end: Node.js

Node.js обрано через його асинхронну модель обробки запитів, що добре підходить для C2C-маркетплейсу з великою кількістю одночасних HTTP-запитів.

Front-end: React.js

React.js обрано для реалізації Single Page Application (SPA), що забезпечує швидкий та динамічний інтерфейс без перезавантаження сторінок. Це важливо для маркетплейсу, де користувачі активно взаємодіють із пошуком, фільтрами, повідомленнями.

СКБД: MySQL

MySQL обрано як реляційну базу даних через:

- чітку структуру даних (користувачі, оголошення, повідомлення, відгуки);
- підтримку транзакцій (важливо для операцій між користувачами);

Для маркетплейсу структура даних є відносно стабільною та добре підходить до реляційної моделі.

2. Архітектуру моделювання за методологією C4 (C4 Model)

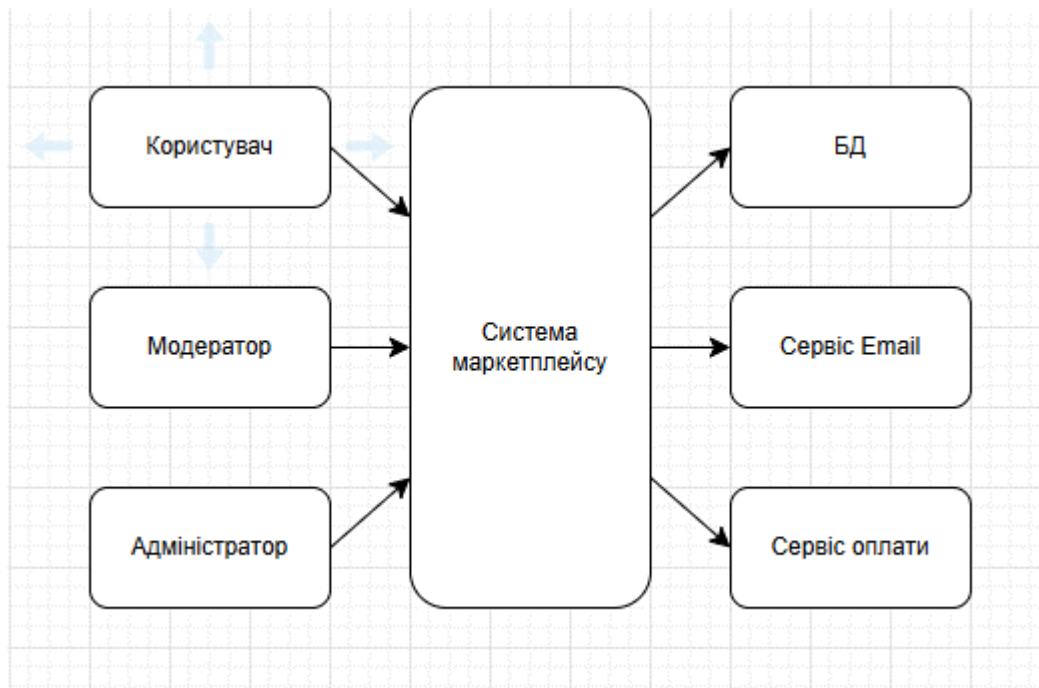


Рис. 1. – Контекстна діаграма

Користувачі взаємодіють із системою через веб-інтерфейс. Система зберігає дані в MySQL та може взаємодіяти із зовнішніми сервісами.

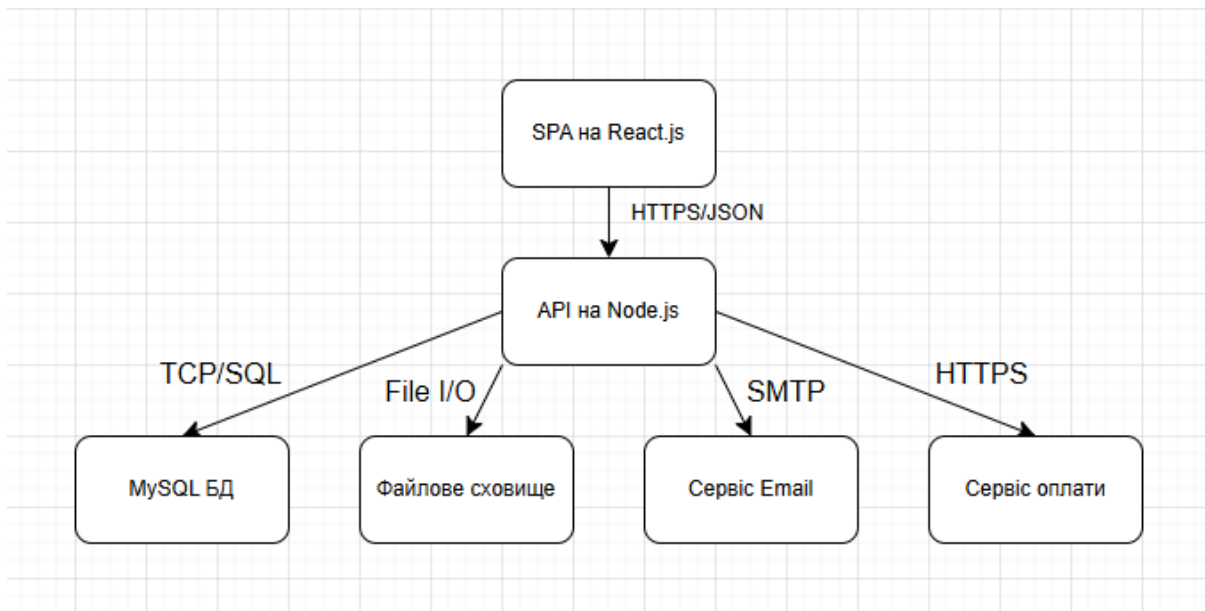


Рис. 2. – Діаграма контейнерів

React SPA звертається до Node.js API через HTTPS. API працює з MySQL та файловим сховищем. Email і Payment – зовнішні сервіси.

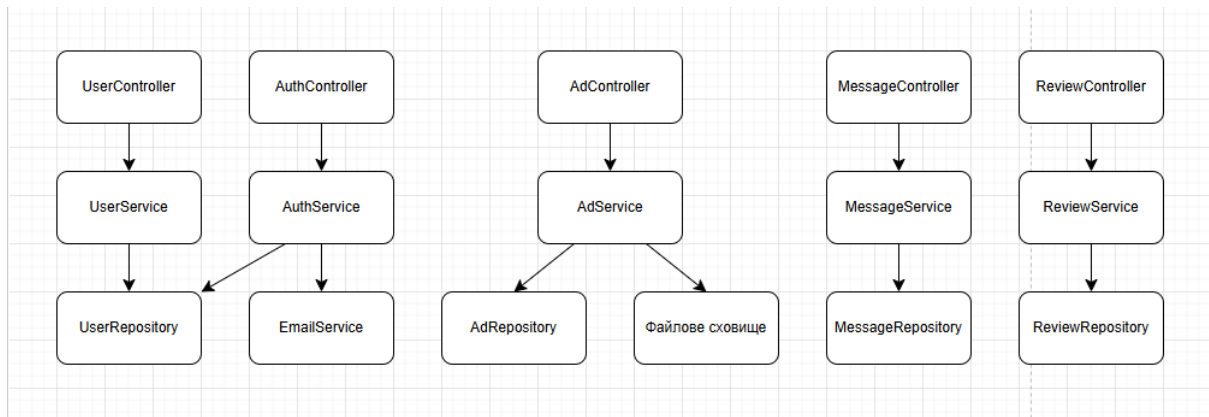


Рис. 3. – Компонентна діаграма

Controllers отримують запити від SPA. Services обробляють бізнес-логіку. Repositories працюють з базою даних. Додаткові сервіси підключаються через сервісний шар.

3. Проектування бази даних (ER-Model / Data Schema)

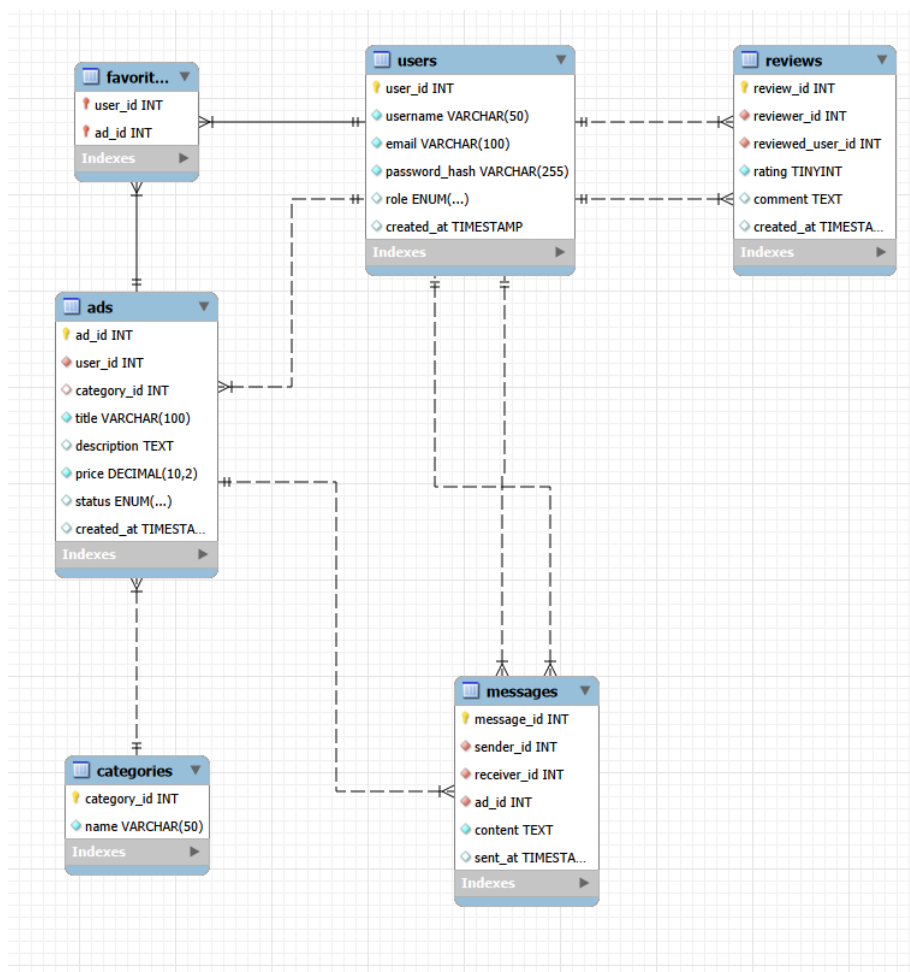


Рис. 4. – ER-діаграма БД